Formal Verification of Lightning Network Multi-Hop HTLC Routing: From Concrete Models to Arbitrary Path Lengths

First Author
University XYZ
City, Country
email@university.xyz

Second Author
University XYZ
City, Country
email2@university.xyz

Third Author
Institute of Technology, ABC
City, Country
email@abc.edu

ABSTRACT

The Lightning Network (LN) relies on Hashed Time-Locked Contracts (HTLCs) and complex channel state machines to enable scalable, multi-hop blockchain payments. Prior symbolic analyses have targeted either two-party HTLC settings or generic message-forwarding abstractions; the interaction between the channel revocation lifecycle, multi-hop routing, and block-height timing has not been mechanized as a single symbolic model. We present such a model in the Tamarin prover, structured as a modular package of five theories establishing 61 distinct machine-checked properties (87 lemma checks in total, as 26 properties are additionally re-proved over the N-hop abstraction), covering the two-party channel lifecycle, multi-hop HTLC routing with fees, staggered-CLTV timing, and an N-hop abstraction.

Our central finding is a consequence of that integration. We identify a *linear single-spend discipline*: two independent soundness gaps—HTLC output duplication and a catastrophic revoked-commitment replay—share one root cause, namely an incorrect mapping of one-shot on-chain resources to Tamarin persistent facts rather than linear ones. Both are repaired by the same structural remedy. The second instance is visible only when revocation and routing are modeled together, which narrower prior models structurally could not observe; we propose the resulting invariant as a reusable modeling discipline for symbolic payment-channel analyses. We further prove end-to-end value conservation, expose a griefing denial of service, establish assumption minimality empirically (showing that removing the intermediary-claim restriction yields a concrete 59-step counterexample), and reproduce the known wormhole attack as a fidelity check. Finally, we identify Tamarin's origin-analysis recursion on self-referential signed messages as a barrier to unbounded verification, and provide a tractability workaround that transfers core routing state-machine invariants to paths of arbitrary length.

CCS CONCEPTS

• **Security and privacy → Formal methods and theory of security**; *Cryptography*.

Conference'17, July 2017, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM
https://doi.org/10.1145/0000000.0000000

KEYWORDS

Formal verification, Tamarin prover, Lightning Network, Payment Channels, HTLC, Blockchain, Layer-2 scaling

ACM Reference Format:
First Author, Second Author, and Third Author. 2027. Formal Verification of Lightning Network Multi-Hop HTLC Routing: From Concrete Models to Arbitrary Path Lengths. In . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/0000000.0000000

1 INTRODUCTION

Blockchain technologies have revolutionized decentralized finance, but their inherent reliance on global consensus creates a fundamental scalability bottleneck. To achieve the transaction throughput required for global adoption, Layer-2 (off-chain) scaling solutions have become essential. The Lightning Network (LN) [14] is the most prominent of these, enabling fast, low-fee payments by moving the bulk of transactions off-chain and settling only disputes on the underlying blockchain [6].

At the heart of the Lightning Network are Payment Channels and Hashed Time-Locked Contracts (HTLCs) [1]. HTLCs enable multi-hop payments across a network of untrusted intermediaries by conditioning the release of funds on the knowledge of a secret preimage, combined with staggered absolute timelocks (CLTV). Alongside this, the channel lifecycle relies on a complex revocation and punishment mechanism to prevent participants from publishing outdated, fraudulent channel states [8].

While the conceptual design of LN multi-hop routing is elegant, the interaction between cryptographic primitives, block-height timing constraints, and off-chain state machines is highly non-trivial. Subtle edge cases—such as early-timeout races where an incoming channel refunds before an outgoing channel is redeemed—can compromise the safety of honest intermediaries. Furthermore, security in these systems is heavily dependent on liveness assumptions: honest nodes must monitor the network and act before their HTLCs expire.

Existing formal treatments have addressed parts of this stack in isolation. Symbolic analyses in Tamarin have targeted two-party HTLC constructions such as cross-chain swaps [2] and contingent payments [3]; generic symbolic frameworks have studied path integrity and agent-skipping in message-forwarding protocols [15]; and model checking with TLA+ has verified balance security for Lightning channels and bounded-length multi-hop payments [16]. What has not been mechanized is the combination: the two-party revocation and punishment lifecycle together with multi-hop HTLC routing, per-hop fees, and staggered CLTV timing, in one symbolic model under a full Dolev–Yao adversary. To our knowledge, the model we present is the first Tamarin formalization to integrate these layers.

That integration is not merely a matter of coverage; it is what makes our central finding observable. We identify an incorrect engineering mapping—modeling one-shot on-chain resources as persistent rather than linear Tamarin facts—that manifests twice, once in the HTLC layer and once in the revocation layer. The second instance involves a rule gated by revocation state being fired repeatedly along a routing execution, and is therefore invisible to models that contain only one of the two layers.

To conduct this analysis we use the Tamarin prover [12] under a full Dolev–Yao adversary [5]. We explicitly model integrity and safety; we do not model payment privacy or unlinkability (a confidentiality property requiring onion routing and observational-equivalence proofs), stated here as an explicit exclusion. Comprehensively verifying the Lightning Network introduces severe methodological and tooling hurdles. First, the protocol inherently requires the combination of Dolev-Yao cryptographic reasoning, natural-number induction (for CLTV arithmetic), and unbounded block clocks. As we demonstrate, attempting to verify these aspects in a single monolithic theory causes state-of-the-art provers to exhaust available memory (OOM). Second, while prior analyses often fix the routing path to a specific length, a natural question arises: do these safety properties hold for paths of arbitrary length (N hops)?

In this paper, we present a comprehensive, modular formal verification of Lightning Network multi-hop HTLC routing. Our contributions are as follows:

(1) **A linear single-spend discipline:** We identify a category error in mapping on-chain outputs to persistent facts, which resulted in two independent soundness gaps (HTLC output duplication and a catastrophic revoked-commitment replay allowing unbounded punishment). Both are repaired uniformly by structural linear-resource guards. We propose the resulting invariant—every one-shot on-chain output is a linear fact—as a reusable modeling discipline for symbolic payment-channel analyses.
(2) **A modular, fully machine-checked model** integrating the two-party channel lifecycle (including revocation and punishment) with multi-hop HTLC routing and fees. It establishes 61 distinct properties; the total of 87 lemma checks includes 26 properties re-proved over the N-hop abstraction, and we report this composition explicitly in Table 1 rather than quoting a single aggregate.
(3) **An empirical assumption-minimality result** proving that removing the active intermediary-claim restriction (T3) yields a concrete 59-step counterexample, together with a two-sided certificate that this specific liveness assumption is load-bearing.
(4) **Value conservation and fidelity checks:** We prove end-to-end value conservation, exhibit a griefing denial of service, and reproduce the known wormhole (agent-skipping) attack [10, 15] as a fidelity check demonstrating that our strict single-spend fix does not over-constrain the protocol.
(5) **A tractability map and workaround:** We identify the precise bounding factors (the signing/nat/clock OOM triple, signed-message N-hop routing) and provide a tractability workaround—abstracting the network layer into linear facts—that allows us to verify the N-hop internal state machine, explicitly noting the trade-off in adversarial network reasoning.

2 RELATED WORK

2.1 Formal Verification of Payment Channels

Early formalizations with idealized channel closures [4, 9, 13] successfully proved payment atomicity and balance conservation but omitted the revocation and punishment mechanisms required to secure real-world implementations.

Prior symbolic analyses have applied Tamarin to HTLC-based cross-chain swaps [2] and zero-knowledge contingent payments over a public ledger [3]. These models target two-party settings and lack multi-hop routing, channel lifecycle revocation, or staggered CLTV arithmetic. Our work explicitly bridges this gap within a unified, automated framework targeting the multi-hop setting.

2.2 Agent-Skipping and Path Integrity

Smith et al. [15] develop a general symbolic framework, in the multiset-rewriting setting, for agent-skipping attacks on message-forwarding protocols. Their unifying notion is *path integrity*—that a message traverses the intermediaries in the order the initiator intended—and the wormhole [10] appears as one of four motivating instances, alongside routing tunnelling, RAPTOR, and middlebox bypass. Analysing Lightning as two one-way protocols, they find the setup phase path-integrity-satisfying and the unlock phase violated; that violation is the wormhole, reconstructed as an ordering failure.

The relationship to our work is one of complementary abstraction, and the boundary is sharp. Their signature Σ comprises hashing, pairing, symmetric and asymmetric encryption, and signatures over the sorts `msg`, `pub`, `fresh` and `Fact`; it contains no arithmetic and no numeric sort. Path integrity is correspondingly an ordering property: for all $M_i <_\pi M_j$, if M_j forwarded the message then M_i did too. Value never enters the predicate—routing fees are discussed in prose as the real-world consequence that makes path integrity worth having, but no balance, amount, or conservation property is modelled. Consequently the wormhole is establishable there as a *skipped hop*, whereas its *cost* is not expressible.

Two further differences bear on the present work. First, their reserved facts are `Net`, `K`, `Pk`/`Ltk` and `ShKey`, with protocol facts tracking the shape of a message as it is wrapped and unwrapped; no on-chain output—funding, HTLC, or revoked commitment—is represented at all. The single-spend question of Section 6.1 therefore cannot arise in their setting: linearity appears only in the standard role of `Fr` facts guaranteeing freshness. Second, they analyse Lightning as *two separate one-way protocols*, a setup phase in which HTLCs are established and an unlock phase in which the preimage propagates, reporting the former path-integrity-satisfying and the latter violated; treating it as a single protocol is explicitly left to future work. Their treatment is thus the clearest illustration of the decomposition our model deliberately avoids, and of why the revocation-layer defect is not visible under it.

We accordingly claim no novelty for exhibiting the wormhole, and cite them for it. Our increment is narrow but real: because our model carries amounts and per-hop fees, we quantify the loss to exactly the bypassed routing fees (Section 6), and we do so in a model that simultaneously carries channel revocation and on-chain settlement rather than an abstracted forwarding chain. Their framework gives the message-ordering skeleton the attack rides on; ours gives the value semantics that says what it costs.

2.3 Model Checking and Computational Treatments

Grundmann and Hartenstein [16] formalize Lightning in TLA+ and model-check balance security, using stepwise refinement and a time abstraction to make single channels and multi-hop payments checkable separately, scaling to payments over up to four hops with two concurrent payments. Their approach is complementary to ours along two axes. Methodologically, model checking explores a bounded state space with an explicit adversary encoding, whereas symbolic verification in Tamarin reasons over unbounded sessions under an equational Dolev–Yao model; consequently their analysis handles concurrency that we do not, while ours handles unbounded message-derivation capabilities. Substantively, their refinement establishes that the current Lightning specification satisfies balance security; our contribution is orthogonal, concerning a specific modeling pitfall in the symbolic encoding of one-shot on-chain outputs.

Kiayias and Litos [7] provide a composable, game-based (UC) security treatment of the LN at the two-party channel layer; it does not machine-check multi-hop HTLC routing. Our symbolic approach achieves a high degree of automation while explicitly covering multi-hop routing under a full Dolev–Yao adversary. Prior work frequently treats the liveness requirement of intermediaries as an implicit assumption. The Tamarin analysis of HTLC swaps by Boyd et al. [2] surfaced a timing-related hidden assumption (stable relative chain growth), which parallels our finding that liveness assumptions are strictly load-bearing; we provide a mechanized, two-sided certificate for this in a model that includes the full channel lifecycle.

3 PROTOCOL OVERVIEW AND THREAT MODEL

3.1 Payment Channel Lifecycle

A payment channel is established via an on-chain funding transaction. Once opened, parties transact off-chain by exchanging signed commitment transactions. To prevent a malicious party from broadcasting an outdated state, LN employs a revocation mechanism. When transitioning to a new state, parties exchange revocation secrets. If a revoked state is published, the other party can unilaterally punish the cheater and claim the entirety of the channel funds, protected by a relative timelock (CSV). Channels can be closed cooperatively or unilaterally.

3.2 Multi-Hop HTLC Routing

A sender who has no direct channel with the intended recipient routes the payment through a path of intermediaries $S \to F_1 \to \cdots \to F_n \to R$. The difficulty is that no intermediary is trusted: a naive sequence of local payments would allow any F_i to accept funds from its predecessor and never forward them. HTLCs remove this trust requirement by making every hop conditional on a single shared secret, so that the payment either completes along the entire path or completes nowhere.

Lock round. The receiver draws a fresh preimage x, publishes an invoice containing $y = H(x)$ and the amount v, and reveals x to no one. The sender then locks funds on its channel with F_1 under the condition that whoever presents x' with $H(x') = y$ before deadline d_0 may claim the output, and that after d_0 the payer may reclaim it. Each intermediary in turn locks a corresponding HTLC on its outgoing channel under the *same* hash y, deducting a routing fee, so that locked amounts decrease and deadlines tighten toward the receiver:

$$v_0 > v_1 > \cdots > v_n = v, \qquad d_0 > d_1 > \cdots > d_n.$$

No funds move during this round; each HTLC is an additional conditional output in the two parties' current commitment transaction, and is therefore subject to the revocation machinery of Section 3.1. Locking is safe for F_i precisely because it holds an incoming HTLC on the same hash for a strictly larger amount: if its outgoing HTLC is claimed, the secret needed to claim its incoming one is thereby exposed.

Release round. The receiver claims the final HTLC by revealing x to F_n, which learns x in the process. F_n presents x on its incoming HTLC to claim v_{n-1} from F_{n-1}, retaining the difference $v_{n-1} - v_n$ as its fee. The preimage propagates backward in this manner until the sender's HTLC is settled. Because a single secret unlocks every hop, disclosure to any party makes the secret available to all parties upstream; this is the source of the atomicity property verified as `Payment_Atomicity_Under_Liveness`.

Why deadlines must decrease. The staggered timelock structure is not a convention but the sole guarantee of intermediary safety. Consider F_i, holding an incoming HTLC with deadline d_{i-1} and an outgoing one with deadline d_i. If F_{i+1} claims the outgoing HTLC at the latest permitted moment d_i, then F_i learns x at d_i and retains the interval (d_i, d_{i-1}) in which to settle its incoming claim. Were the ordering inverted, so that $d_{i-1} < d_i$, the downstream party could claim at d_i after F_i's incoming HTLC had already expired and been refunded: F_i would have paid v_i downstream while recovering nothing upstream, losing the full forwarded amount. The gap $d_{i-1} - d_i$ is the CLTV delta, and Section 6.4 derives its positivity and the non-emptiness of the resulting claim window from a concrete block clock rather than assuming them.

Residual liveness requirement. A non-empty claim window is necessary but not sufficient, since nothing in the timelock structure compels F_i to use it. An intermediary that is offline during (d_i, d_{i-1}) forfeits its incoming HTLC even though every party followed the protocol. Security at the routing layer therefore reduces, at this point, to a liveness assumption on honest intermediaries, which corresponds in deployment to watchtower infrastructure [11]. Two further pathologies survive an otherwise correct execution: a receiver that never discloses x freezes the capital of every party on the path until the deadlines elapse (a griefing denial of service), and two non-adjacent colluding parties that relay x out-of-band bypass the intermediary between them, which forwards liquidity and earns no fee (the wormhole attack). Both are reachable in our model.

3.3 Threat Model and Security Goals

We adopt the standard Dolev-Yao adversary model [5], representing a network attacker with full control over the communication medium but who cannot break cryptographic primitives. We explicitly model long-term key compromise as a distinct adversarial capability.

The Liveness Assumptions. Security degrades into a liveness requirement. Our model encodes four timing restrictions. We draw a distinction between passive temporal orderings and active node behaviors:

- **Passive Orderings:** T1 (`RedeemBeforeTimeout`), T2a (`ForwardTimeGap`), and T2b (`HonestPartiesActBeforeIncomingTimeout`). These are assumed structural constraints of the CLTV clock.
- **Active Sweeping:** T3 (`IntermediaryMustClaim`). This encodes the active requirement that an intermediary must sweep their incoming HTLC once their outgoing one is redeemed.

As we demonstrate in Section 6, T3 is strictly load-bearing for safety, whereas T1/T2 are load-bearing for timing soundness. This mirrors the real-world necessity of LN watchtowers [11]. Our core security goals translate directly to verified lemmas: Payment Atomicity, Intermediary Safety, Routing Causality & Authentication, Timing Safety, and Value Conservation.

4 FORMAL PRELIMINARIES

We fix notation for Tamarin's multiset-rewriting semantics [12], which the remainder of the paper uses to state the modeling discipline of Section 6 and the refinement argument of Section 7. Terms and facts are standard. A fact is $F(t_1, \ldots, t_k)$. The set of fact symbols is partitioned as $\Sigma_{\text{fact}} = \Sigma_{\text{lin}} \uplus \Sigma_{\text{per}}$ into linear and persistent symbols; we write persistent facts with a leading bang, e.g. `!RevokedSecret(·)`. A state S is a finite multiset of ground facts.

The critical distinction for our contribution is the transition relation. For a rule $r_i = l \xrightarrow{a} r$ with linear premises $\text{lin}(l)$ and persistent premises $\text{per}(l)$,

$$S \xrightarrow{a} \big(S \setminus^{\#} \text{lin}(l)\big) \cup^{\#} r \quad \text{provided} \quad \text{lin}(l) \subseteq^{\#} S \,\wedge\, \text{per}(l) \subseteq S.$$

Only the linear premises are removed from the state; persistent premises are checked but remain available. A trace is the sequence of action multisets of an execution from the empty state; properties are first-order formulas over $F(\bar{t})@i$ with timepoint ordering $i \lessdot j$. We write $R \models_{\forall} \varphi$ when all traces satisfy φ and $R \models_{\exists} \varphi$ when some trace does. Restrictions filter the trace set and are therefore *assumptions*, not theorems, which is why Section 6 tests whether one of ours is load-bearing.

This leads to a well-known mechanical consequence: if a rule exercising a resource is gated only by a persistent fact, and its other premises are re-derivable, the rule can fire unboundedly. While this is a standard feature of multiset-rewrite systems, our contribution is not the observation of this feature, but rather *identifying that specific Lightning Network semantics (channel outputs and revoked commitments) map exactly to this pitfall*, and demonstrating the concrete, catastrophic exploits that arise if the mapping is done incorrectly. The difficulty is one of engineering judgement rather than of mathematics: a revoked commitment presents itself in the protocol description as *knowledge*—the counterparty learns a secret—and knowledge is precisely what persistent facts are for, so the natural encoding is the wrong one.

5 FORMAL MODELING METHODOLOGY

5.1 The Curse of Monolithism and Modular Decomposition

A naive approach is to construct a single, monolithic theory containing the channel lifecycle, HTLC routing logic, CLTV block-height arithmetic, and value conservation. This is fundamentally intractable. The combination of the signature equational theory, natural-number induction, and an unbounded block clock causes Tamarin's search space to explode. Crucially, this was tested, not assumed—any two of the three ingredients remain tractable; only the full triple OOM-kills the prover.

The model is therefore split so that no single theory carries all three. The architecture is summarized in Table 1.

5.2 Tactic Engineering and Reproduction

All results were reproduced in a single sitting on one machine under Tamarin 1.10.0 with Maude 3.1; every theory passes Tamarin's well-formedness checks, and the whole package verifies in 542.36 s of wall-clock time. Results were produced under a strict test-first discipline: a property is claimed only after Tamarin reports it. Per-lemma invocation is used on the large theory to avoid OOM. The flag `--stop-on-trace=seqdfs` rescues the heavy existence witnesses but hangs the natural-number arithmetic theories, so it is applied selectively.

6 VERIFICATION OF THE CORE MODEL

Across the theories of Table 1 we establish 61 distinct properties, corresponding to 87 lemma checks. Of the N-hop theory's 31 lemmas, 26 restate properties already proved in the concrete theory over the abstract routing layer and 5 are new to it (Section 7). Of the 43 lemmas of the concrete theory, 10 are exists-trace witnesses (including non-vacuity checks and the attack traces below) and 5 are `[reuse]` helper lemmas supporting other proofs rather than stating protocol goals; the complete lemma index accompanies the artifact. We focus here on the concrete theory `multihop.spthy`.

6.1 The Linear Single-Spend Discipline

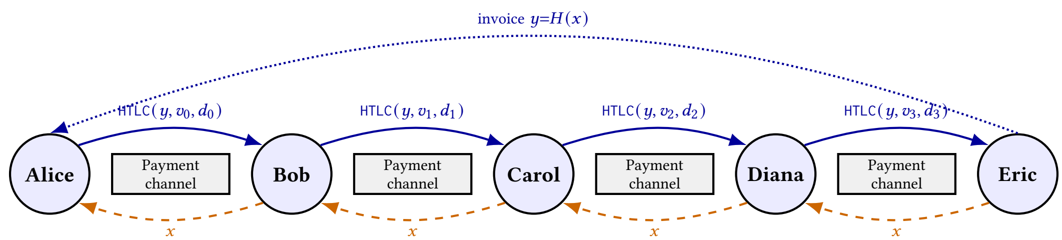
We now state the modeling discipline that resolves the soundness gaps.

Definition 6.1 (One-shot resource). A fact symbol `Res` denotes a one-shot resource if, in the intended protocol semantics, the privilege represented by $\texttt{Res}(\bar{t})$ may be exercised at most once. On-chain outputs—a funding output, an HTLC output, the output of a revoked commitment—are one-shot resources.

Definition 6.2 (Linear single-spend token). A single-spend token for a one-shot resource is a linear fact symbol $\texttt{Tok} \in \Sigma_{\text{lin}}$ together with a discipline requiring (i) every rule exercising the resource to

Table 1: The five-theory modular architecture. The first four theories establish 56 distinct properties, and the N-hop theory adds a further 5, for 61 in total. Of the N-hop theory's 31 lemmas, 26 restate properties already proved in `multihop.spthy` over the abstract routing layer and 5 are new to it (generic forwarding causality and the unbounded-path value lemmas), giving 87 lemma checks in total.

Theory File	Aspect Modeled	Safety	Witness	Time	Builtins
`multihop.spthy`	Lifecycle, 3-hop routing, value/fees, wormhole	33	10	353.31 s	Hashing, Signing, Nat
`Clock.spthy`	Block-clock lifecycle & staggered-CLTV claim windows	6	3	54.37 s	Natural-numbers
`Cltv.spthy`	Pure CLTV block arithmetic (delta positivity)	3	0	0.23 s	Natural-numbers
`timeout.spthy`	Early-timeout race (T2b removed)	0	1	0.06 s	—
Distinct properties		**42**	**14**		
`Modif.spthy`	*Re-proof* of the above over the N-hop abstraction	19	6	134.39 s	Hashing, Signing, Multiset



$v_0 > v_1 > v_2 > v_3{=}v, \quad d_0 > d_1 > d_2 > d_3$

Figure 1: A multi-hop HTLC payment routed over a path of payment channels. Solid blue arrows: the lock round, each hop offering an HTLC with hash-lock y, value v_i, and deadline d_i. Dashed orange arrows: the release round, the preimage x propagating backward once revealed.

take Tok($\bar{t}$) as a linear premise, and (ii) Tok($\bar{t}$) to be produced at most once per $\bar{t}$ across all rules.

Under Definition 6.2 the number of consumptions of Tok($\bar{t}$) is bounded by the number of its productions, which is at most one; single spend is therefore a structural consequence of linearity rather than an axiom. Concretely, `Free(ptr)` is produced once by `Lock_Funds_And_Open` (one per channel open) and consumed by every HTLC-add. Furthermore, `RevokedCommitmentUnspent(ptr, A, B, n, commit)` is produced once by `Revoke_Old_Secret` and consumed by `Publish_Revoked`:

```
Publish_RevokedA :
[ !ChannelRecord(ptr, A, B), !ValidOnChain(ptr),
  !RevokedSecret(A, n, s, c),
  RevokedCommitmentUnspent(ptr, A, B, n, c) ]
--[ PublishRevoked(A, n, c), Cheat(A, n) ]->
[ OnChainCheat(A, B, n, c), RevealedSecret(A, n, s, c) ]
```

Persistent knowledge of the revocation secret is retained via `!RevokedSecret`; only the one-shot spend is linear. This separation is the content of the discipline.

Instance 1: HTLC output duplication. The forwarding rules were gated only by a public HTLC-offer message. A Dolev–Yao replay could re-fire an honest forward, locking a second HTLC on the same channel output with an adversary-chosen amount. We replaced a global `OneHTLCPerPtr` axiom with the structural `Free(ptr)` guard.

Instance 2: Catastrophic revoked commitment replay. The original punishment rules were gated only by the persistent `!RevokedSecret` fact. Under the transition relation of Section 4, this allowed the punishment rule to fire arbitrarily many times for the same revoked state. The consequence is concrete rather than cosmetic: a revoked commitment corresponds to one on-chain transaction spending one funding output, yet a single instance of cheating licensed `PublishRevoked`—and hence `Punish`—without bound, so the model admitted executions in which the honest party confiscates the channel balance repeatedly from one cheat. The defect is silent because every individual lemma about punishment soundness continues to hold: `No_Punishment_Without_Cheating` is unaffected, since punishment still requires a prior cheat. Only the *multiplicity* is wrong, and multiplicity is exactly what no single safety lemma in the original formulation constrained.

We stress that Instance 2 is a consequence of integration rather than of scale. It resides at the junction of the revocation lifecycle and the routing execution. A model containing only the two-party channel layer does not exhibit the trace that exposes it.

Re-verification. Adding a linear premise shrinks the enabling condition, so traces(R') $\subseteq$ traces(R) and every all-traces ($\forall$) property is preserved automatically. The converse fails for existence ($\exists$) witnesses, which may be removed. We therefore re-ran the full 43-lemma theory rather than appealing to monotonicity; all 43 re-verify (353.31 s).

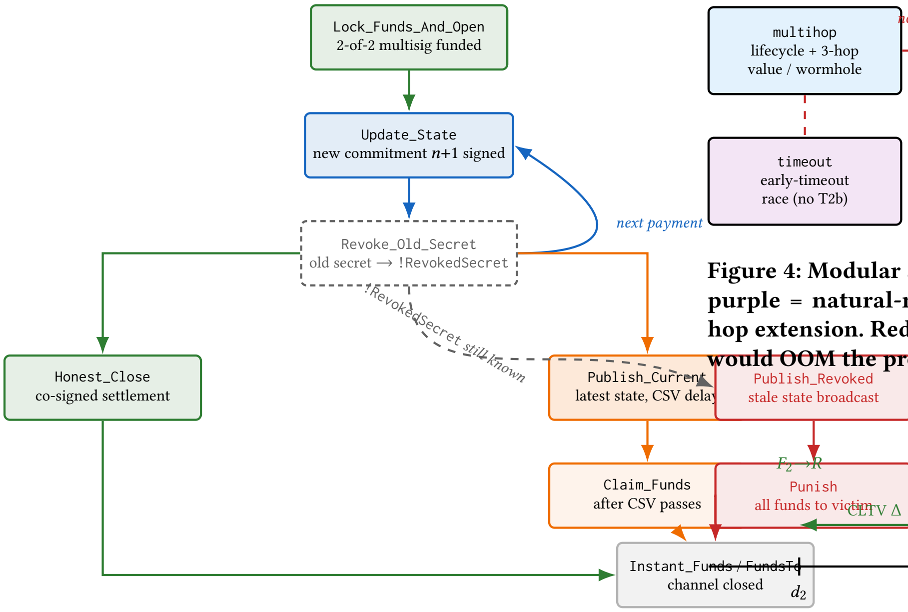


Figure 2: The full channel lifecycle. Green: open and cooperative close. Blue: state update. Gray dashed: the revoked secret, persistent and re-checkable on every cheat attempt. Amber: honest unilateral close with its CSV delay. Red: publishing a revoked state and the resulting punishment. All four paths converge on settlement.

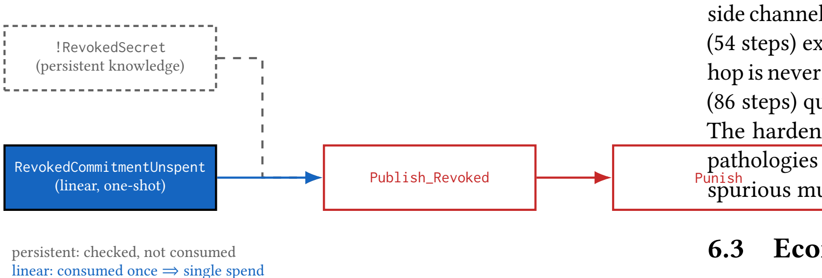


Figure 3: Revocation secret stays persistent (gray); the on-chain spend is linear (blue), consumed en route to Punish (red).

6.2 Wormhole Attack: A Fidelity Check

The wormhole attack [10] is an instance of the agent-skipping class formalized symbolically by Smith et al. [15], and we claim no novelty for exhibiting it. We do, however, quantify it: their formalism carries no numeric sort, so the magnitude of the loss is not expressible there, whereas our value layer pins it to exactly the bypassed fees. Beyond that increment, its role here is to discharge an obligation created by Section 6.1: having constrained the model with two linear single-spend tokens, we must show the repair is *sound* rather than merely restrictive, since a guard that eliminated the unbounded-punishment trace by over-constraining the protocol would also eliminate legitimate executions and render subsequent safety results vacuous.

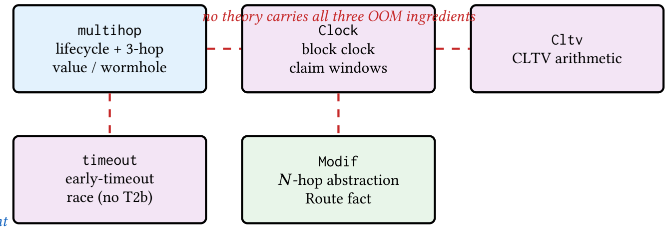


Figure 4: Modular architecture: blue = concrete crypto core, purple = natural-number timing theories, green = the N-hop extension. Red dashed edges mark pairs that, combined, would OOM the prover.

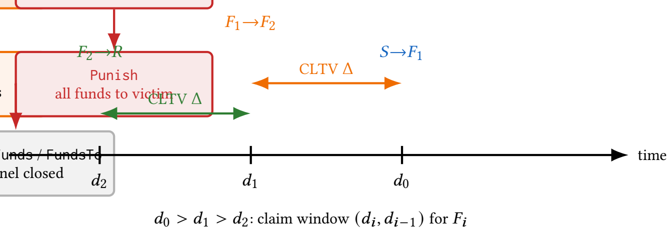


Figure 5: Staggered CLTV deadlines along the path, colored to match Figure 1's hops.

In the symbolic model, the Dolev–Yao network is the colluders' side channel. We machine-check it twice: `Wormhole_Fee_Theft_Reachable` (54 steps) exhibits a trace where the payment settles yet the middle hop is never redeemed, with no key compromise; and `Wormhole_Steals_Exactly_` (86 steps) quantifies the theft to precisely the path fees `%f1 + %f2`. The hardened model therefore still admits the known economic pathologies of HTLC routing: the linear tokens remove exactly the spurious multiplicity and nothing else.

6.3 Economic Soundness and Limitations

We prove `EndToEnd_Value_Conservation`: the sender locks the receiver's amount plus the accumulated fees, so the routing mechanism neither creates nor destroys value. As a reachability result, `Honest_Intermediary_Refunds` exhibits an honest intermediary induced to lock capital and then refund on timeout, with no settlement and no key compromise—the standard PCN griefing denial of service. This is not a violation of safety, since principal is recovered, but it is an availability concern: an adversary can immobilize routing capital at will.

Remark 1 (Zero-fee forwarding is out of scope). The Tamarin `natural-numbers` sort lacks a zero term (`%0`). Consequently a forwarding rule requiring `Eq(%vIn, %vOut + %fee)` structurally prevents zero-fee forwards, and the lemmas `Fee_Strictly_Positive_Hop1/2` hold for a reason that is an artifact of the encoding rather than a property of Lightning. We therefore do not present them as economic theorems. Because deployed Lightning explicitly supports zero-fee routing for channel rebalancing and probing, our model formally verifies a *restricted subset* of the protocol in which every hop charges a positive fee. We note that value conservation and atomicity do not depend on fee positivity; only the fee-specific lemmas do. Extending the sort to admit zero is left to future work.

6.4 Timing Safety and Assumption Minimality

`cltv_blocks` derives the CLTV-delta invariant from concrete block numbers; `Clock` adds block-clock lifecycle safety; and `timeout` exhibits the early-timeout race that reappears once T2b is removed. Together with `Timeout_Race_Blocked` in the concrete theory, the last of these forms a two-sided certificate: the race is reachable without the assumption and unreachable with it.

We tested whether the active sweeping restriction T3 is redundant given the passive orderings T1/T2b. We removed T3 and re-ran the suite: `Intermediary_Never_Loses_Under_Liveness` is falsified, with Tamarin finding a concrete 59-step counterexample. T2b orders the two timeouts but does not force the node to sweep the incoming HTLC; that action is exactly what T3 encodes, and no ordering constraint can compel an action. We have empirically demonstrated that T3 is load-bearing for intermediary safety. We tested the removal of T3 specifically because it is the only one of the four that is an obligation rather than an ordering, and accordingly make no claim regarding the individual minimality of T1, T2a, or T2b, whose content is independently corroborated by the block-clock derivation above.

7 SCALING TO N-HOPS: A TRACTABILITY WORKAROUND

Replacing the hardcoded forward rules with one generic rule was attempted in several encodings. A generic signed-message forward causes Tamarin to OOM even for a 1-hop witness due to self-referential origin-analysis recursion.

To bypass this, we abstract the network layer into an internal Tamarin linear fact—`Route(prev, me, ptr, y)`—handed directly from one hop to the next. Because the HTLC is no longer a self-referential signed term, Tamarin's origin analysis no longer recurses. This yields `Modif.spthy`, which verifies 31 lemmas in 134 s. Carrying the routed amount inside the `Route` fact—so that hop i's outgoing amount *is* hop i+1's incoming amount—makes the fee arithmetic chain along a path of any length, and the receiver's rule closes the chain by checking the arriving amount against its invoice. Consequently `Amount_Strictly_Decreases_Per_Hop`, `Receiver_Paid_Invoice_Amount` and `Fee_Only_On_Successful_Forward` hold at unbounded N, as does `Forward_Requires_Incoming` (279 steps), which states that every forward is preceded by the offer or by the predecessor's forward *with agreeing amounts*. A single lemma, `Settle_Requires_Receiver_Release`, remains commented out: resolving `SenderSettled` backwards walks the whole generic settle chain and does not terminate. It is the one genuine tractability boundary of this layer.

We note that adding the two linear single-spend tokens to this model *reduced* its verification time from 265 s to 134 s. This is the expected direction—a linear premise shrinks the enabling condition and therefore the search space—but it is worth recording, because the intuition that extra facts make a theory harder is what had previously discouraged us from combining the two.

Honest Scope of the Abstraction. By internalizing the network into a `Route` fact, we explicitly remove the Dolev–Yao network adversary from the N-hop model. The adversary can no longer inject, replay, or forge messages. Therefore, this abstraction does *not* prove that unbounded-length paths are secure against network-level attacks such as the wormhole, which is structurally unreachable here by design. Instead, it verifies that the *internal state machine* of N-hop routing—the cascading timeouts, the preimage propagation logic, and the local state transitions—remains sound for arbitrary N.

Value conservation is not merely a firing guard here. Because `Route` carries the amount, the per-hop split is chained across the path, and the receiver accepts only its invoice amount; the value lemmas above are therefore genuine properties of the unbounded model rather than restatements of a rule condition. What remains out of reach is a *single closed-form* statement summing N fees, which would require quantifying over N inside one formula.

7.1 Refinement Theorem

We now state what transfers from the abstract model A to the concrete model C, and under which hypothesis.

Definition 7.1 (Shared alphabet and projection). Let

$$\Sigma = \{\,\mathsf{ForwardHTLC},\ \mathsf{Redeemed},\ \mathsf{Refunded},\ \mathsf{LtkCompromised}\,\}$$

be the safety-relevant action symbols, emitted with identical arities by both theories. The projection π maps a concrete trace to an abstract one by retaining every Σ-action verbatim, deleting all network and cryptographic events, and relabelling the position-indexed $\mathsf{HTLCForwarded}_i$ to the generic $\mathsf{HTLCForwarded}$.

Definition 7.2 (Auth). C satisfies Auth if every forwarding step is preceded by a genuine offer or forward from the *adjacent* previous hop, unless that hop's key is compromised; for the first position,

$$\begin{aligned}&\forall F_1, F_2, S, p, y, v_{\text{in}}, v_{\text{out}}, i.\ \mathsf{HTLCForwarded}_1(F_1, F_2, S, p, y, v_{\text{in}}, v_{\text{out}})@i \Rightarrow\\ &\quad (\exists p', j.\ \mathsf{HTLCOffered}(S, F_1, p', y, v_{\text{in}})@j \wedge j \lessdot i)\\ &\quad \vee (\exists k.\ \mathsf{LtkCompromised}(S)@k \wedge k \lessdot i),\end{aligned}$$

and symmetrically for the second position.

Auth is machine-checked in C as `Forward1_Requires_Offer` (270 steps) and `Forward2_Requires_Forward1` (278 steps). It quantifies over a *single adjacent pair* and never over the path or N; this locality is what makes it provable, and its failure for a generic signed rule is precisely the boundary reported above.

Theorem 7.3 (Soundness of the abstraction). *Let φ be a safety property over Σ with $A \models_\forall \varphi$ for all path lengths N, and assume $C \models_\forall$ Auth. Then $C \models_\forall \varphi$.*

Proof sketch. By induction on the number of forwarding steps of a trace tr of C. An honestly authenticated incoming HTLC is exactly what licenses the abstract `Route` hand-off, so each concrete forwarding step projects to a legal abstract step; the forged or replayed case is excluded by Auth except under key compromise, which both models expose identically via $\mathsf{LtkCompromised} \in \Sigma$. Hence $\pi(tr)$ is an abstract trace. As π preserves Σ-actions verbatim and φ is over Σ, any violation in tr appears in $\pi(tr)$, contradicting $A \models_\forall \varphi$. ◻

Remark 2 (Scope of mechanization). Both hypotheses of Theorem 7.3 are machine-checked—Auth in C, and φ in A—but the composition is pen-and-paper: Tamarin can neither quantify over N nor relate two theories, for the same reason that key-management meta-theorems are argued externally. Soundness rests on the locality of Auth, which we confirmed by inspecting every forward rule: each references only its adjacent pair. This is the honest boundary of our verification.

8 DISCUSSION: WHAT FAILED AND TRACTABILITY BOUNDARIES

We record negative results explicitly—they are a core methodological contribution.

- **The Three-Ingredient OOM.** Signing + natural-number induction + an unbounded block clock together exhaust the prover; any two are fine. This is the reason for the modular split.
- **Signed-Message N-Hop Routing.** Origin analysis on the self-referential `'htlc'` term does not terminate, even for reachability, even with a fresh ID.
- **A Rejected Restriction.** An initial fix `OneRedeemPerPtr` ("at most one redeem per output") was tested and rejected: it falsified the honest witness `Multihop_Payment_Possible`, because the two `Redeemed` events in an honest run are the two endpoints' views of one hop, not two spends. The correct guard was `OneHTLCPerPtr` (later made structural).
- **A Caught False Positive.** An exists-trace probe sharing only the payment hash appeared to show "value creation". Tightening it to a single linked payment chain falsified it—the apparent attack was invoice-hash reuse across two unrelated offers, not a soundness bug.
- **Reusable Channels (Boundary).** Stage 1 (structural single-HTLC-per-output via the `Free(ptr)` slot) ships and re-verifies 43/43. Stage 2 (return `Free(ptr)` on resolution) is a clean tractability boundary: all-traces safety lemmas verify (~45 s each), but every exists-trace witness fails to converge. Stage 1 is therefore the shipped model.

9 CONCLUSION AND FUTURE WORK

We delivered a modular, machine-checked model that integrates the Lightning two-party channel lifecycle with multi-hop HTLC routing and fees, establishing 61 distinct properties across five theories. That integration surfaced a linear single-spend discipline: two independent soundness gaps arising from encoding one-shot on-chain outputs as persistent facts, repaired uniformly by linear tokens, and generalizable into an auditable modeling invariant. We further established the minimality of the T3 timing assumption empirically, proved value conservation, exhibited a griefing denial of service, reproduced the known wormhole attack as a fidelity check, and mapped where the prover stops scaling.

The protocol-level safety results confirm expectations—consistent with the composable cryptographic treatment of Lightning [7] and the model-checking results of Grundmann and Hartenstein [16], which our symbolic model complements at the routing layer. The durable contribution here is methodological—a modeling discipline that catches a class of silent encoding errors, together with an honest account of what it takes to keep Tamarin tractable on a stateful, time-locked, arithmetic protocol.

Future work includes testing whether the persistent-versus-linear pitfall is latent in other published symbolic models of payment channels, which would establish the discipline as a class-level finding rather than a single-model observation; extending the framework to Point Time-Locked Contracts (PTLCs); and bridging to computational verification. Our value conservation proofs verify strict amount invariants, but do not enforce game-theoretic economic rationality (e.g., defending against timelock bribery), which remains outside symbolic reasoning.

REFERENCES

[1] Lightning Network community. 2024. BOLT: Basis of Lightning Technology (Lightning Network Specifications). https://github.com/lightning/bolts.

[2] Colin Boyd, Kristian Gjøsteen, and Shuang Wu. 2020. A Blockchain Model in Tamarin and Formal Analysis of Hash Time Lock Contract. In *2nd Workshop on Formal Methods for Blockchains (FMBC)*, 5:1–5:13.

[3] Sergiu Bursuc and Steve Kremer. 2019. Contingent Payments on a Public Ledger: Models and Reductions for Automated Verification. In *European Symposium on Research in Computer Security (ESORICS)*. Springer, 361–382.

[4] Christian Decker and Roger Wattenhofer. 2015. A Fast and Scalable Payment Network with Bitcoin Duplex Micropayment Channels. In *Stabilization, Safety, and Security of Distributed Systems (SSS)*. Springer, 3–18.

[5] Danny Dolev and Andrew C. Yao. 1983. On the Security of Public Key Protocols. *IEEE Transactions on Information Theory* 29, 2 (1983), 198–208.

[6] Lewis Gudgeon, Pedro Moreno-Sanchez, Stefanie Roos, Patrick McCorry, and Arthur Gervais. 2020. SoK: Layer-Two Blockchain Protocols. In *Financial Cryptography and Data Security (FC)*. Springer, 201–226.

[7] Aggelos Kiayias and Orfeas Stefanos Thyfronitis Litos. 2020. A Composable Security Treatment of the Lightning Network. In *IEEE 33rd Computer Security Foundations Symposium (CSF)*. IEEE, 334–349.

[8] Matteo Maffei. 2024. *Cryptocurrencies, Lecture 11: Scalability (Payment Channels and Payment-Channel Networks)*. Lecture Notes.

[9] Giulio Malavolta, Pedro Moreno-Sanchez, Aniket Kate, Matteo Maffei, and Srivatsan Ravi. 2017. Concurrency and Privacy with Payment-Channel Networks. In *ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM, 455–471.

[10] Giulio Malavolta, Pedro Moreno-Sanchez, Clara Schneidewind, Aniket Kate, and Matteo Maffei. 2019. Anonymous Multi-Hop Locks for Blockchain Scalability and Interoperability. In *Network and Distributed System Security Symposium (NDSS)*. The Internet Society.

[11] Patrick McCorry, Surya Bakshi, Iddo Bentov, Sarah Meiklejohn, and Andrew Miller. 2019. Pisa: Arbitration Outsourcing for State Channels. In *ACM Conference on Advances in Financial Technologies (AFT)*. ACM, 16–30.

[12] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. 2013. The TAMARIN Prover for the Symbolic Analysis of Security Protocols. In *Computer Aided Verification (CAV)*. Springer, 696–701.

[13] Andrew Miller, Iddo Bentov, Surya Bakshi, Ranjit Kumaresan, and Patrick McCorry. 2019. Sprites and State Channels: Payment Networks that Go Faster Than Lightning. In *Financial Cryptography and Data Security (FC)*. Springer.

[14] Joseph Poon and Thaddeus Dryja. 2016. The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments. https://lightning.network/lightning-network-paper.pdf. Whitepaper.

[15] Zach Smith, Hugo Jonker, Sjouke Mauw, and Hyunwoo Lee. 2022. Modelling Agent-Skipping Attacks in Message Forwarding Protocols. arXiv:2201.08686.

[16] Matthias Grundmann and Hannes Hartenstein. 2025. Model Checking the Security of the Lightning Network. arXiv:2505.15568.